The Journal of Open Source Software

Projit: An Open Source tool for Decoupled Data Science

John Hawkins [1,2*]

1 Transitional AI Research Group, **2** Pingla Institute, Sydney, Australia * These authors contributed equally.

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor:

Submitted: 10 November 2024
Published: unpublished

License
Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

Summary

Data science projects occupy an unsual space between rapid scripting, software development, and methodologically rigorous experimentation. They require careful discipline to prevent subtle problems like target leakage, over-fitting or p-hacking. At the same time they cannot deliver results if they are locked down by rigid frameworks. Typically, data scientists use custom workflows, or proprietary cloud systems to automate and standardise certain elements like management of data sets, scripts, model artefacts and experimental results. The general absence of standardisation means that we cannot easily migrate projects or audit them without significant investment in understanding a codebase, nor can we easily repeat experiments or conduct meta-analysis across multiple projects. We present `projit` – a simple open source package and CLI for maintaining data science project meta-data and interoperability between stages and processes. https://github.com/john-hawkins/projit

Statement of need

Software approaches to managing scientific data, processes and meta-data are typically either built as front-ends for specific scientific domains (Howe et al., 2008; Pettit et al., 2010) or they are designed to facilitate interoperability between different technology stacks (Subramanian et al., 2013). Machine learning project frameworks tend to solve problems of model training and deployment for specific technologies (Alberti et al., 2018; Domenech & Guillén, 2020), and hence have limited application for general data science work.

`Projit` is a Python package for managing data science project meta-data inside a simple local JSON store. It provides a CLI tool for interrogating this data so that the project can easily be assessed and understood. The API for `projit` was designed so that the package can be included in python scripts to locate datasets, register experiments and store results along with hyper-parameters.

The `projit` datastore is light-weight so it can be saved with code inside a source code repository, allowing future users to interrogate the experiment history of a project. This is useful for project continuation, auditing/repeatability and opening the possibility of scripted meta-data analysis. The authors have used the `projit` package in multiple published research projects to manage the results of machine learning experiments into biomedical literature reviews (Hawkins & Tivey, 2024) and the analysis of text features derived from URLS (Hawkins, 2023). In addition, `projit` has been used by the author for commercial projects within industry to benchmark machine learning techniques for classification and regression problems.

DRAFT

The Journal of Open Source Software

Methodology

The core design principle of `projit` is that data science projects should be structured as loosely coupled components, with shared meta-data. Some dependency is inevitable, but it should be kept to an absolute minimum. For example, experiments depend on a data processing pipeline, but do not need to depend on anything but the output of that process. Experiments should be able to be executed in parallel, so that they can be re-run as required. They do not need to be aware of each other, but they should generate standardised result sets for comparison.

To facilitate loose coupling between stages of the project the `projit` utility imposes a simple schema for components of a data science project. These consist of:

- Datasets
- Experiments
  - Executions
- Results

These entities can be added, removed or modified using either the CLI tool or the Python package within scripts. A standard project workflow is depicted in Figure Figure 1. On the right hand side of the figure you see the common sequence of critical stages in standard data science (or data mining) workflow models like the CRISP-DM(Wirth & Hipp, 2000). Each of these stages depends on entities defined and created in previous steps. Use of the `projit` meta-data repository facilitates loose coupling between steps. On the left hand side we depict additional meta-analysis that can be performed across multiple meta-data stores.

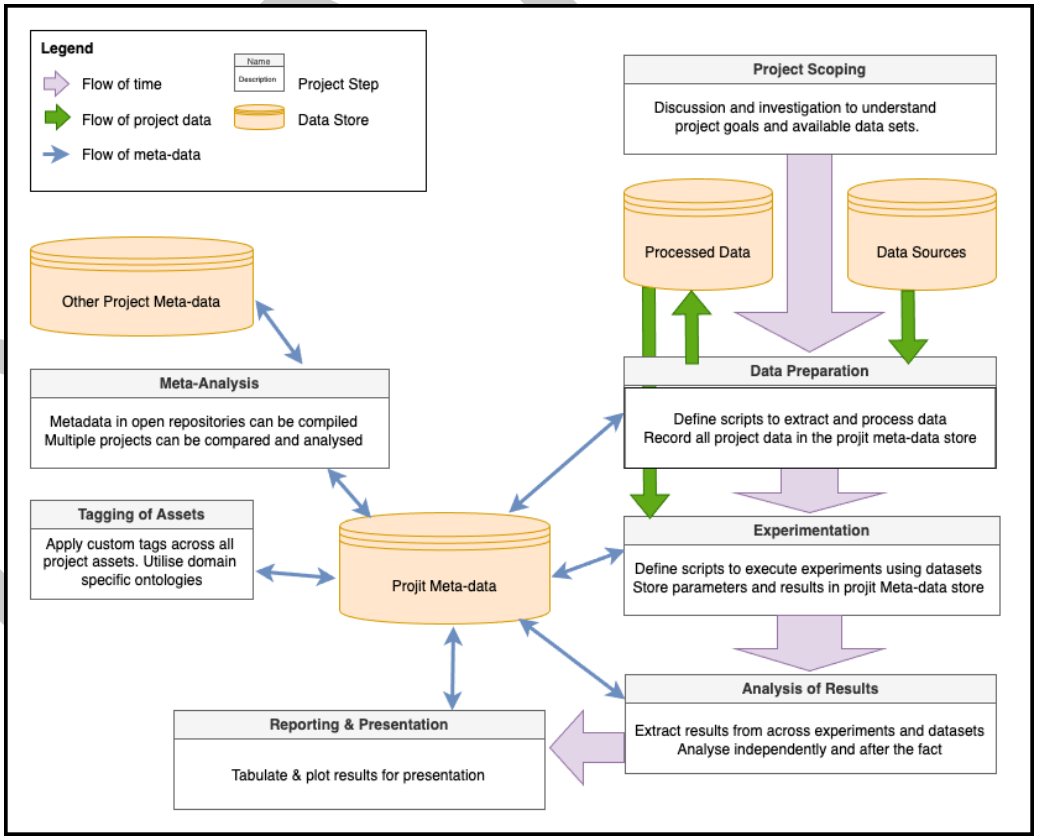

Figure 1: Projit Application Workflow & Usage.

In the development of `projit` we have drawn on additional design principles from other open source projects, including the Git CLI (Torvalds, 2005) and Cookie Cutter Data Science

(Paolino, 2020), discussed in the sections below.

Project Structure

Configuration allows users to determine a standard project structure. This option will initialise any project with a predetermined set of directories and files. We draw upon the principle used in the Cookie Cutter Data Science project when implementing these project structures (Paolino, 2020). However, rather than be prescriptive on project structure we allow it to be customised through configuration.

Natural Language Sub Command CLI

In order to make the CLI interface easy to use we borrowed multiple ideas from the design of the Git CLI (Torvalds, 2005). Firstly, any command will recursively search from the current directory to discover the current project. This means users can run commands from anywhere inside the project without tracking the location of the root directory. Secondly, we develop a sub-command structure that allows the `projit` CLI to be a versatile tool with something close to a natural language interface. For example, the primary command `list` can be applied to any of the `projit` entities, as shown in the command below:

```
> projit list datasets
```

The same principle applies to the remove and add commands, which naturally require additional parameters to specify what is being added or removed. The design goal of the CLI is to make `projit` intuitive without imposing arbitrary constraints.

Research Applications

The fundamental research application of `projit` is in managing the project lifecycle and minimising dependencies between steps. Paths to datasets are retrieved from meta-data, not hard coded. Experiments are named, with execution times tracked. Results of all experiments can be tracked over each iteration, with hyper-parameters and interrogated to easily produce tables of data and analysis. Additional application comes with a focus on open science, allowing other teams to review and audit experiment history, then easily repeat or extend experiments. Finally, there is a research application in meta-analysis. Projects in which the `projit` meta-data are stored along with open source code can be analysed to look at the performance of certain techniques or algorithms across multiple projects.

Example Usage

The following demonstrates the process of initialising a project, registering a dataset and experiment, then tracking results. First change into the project directory and initialise it as a `projit` project.

```
> projit init <Project-Name> template=default
```

The `template` variable is optional and only used if you want to create a directory structure. Next step is to add one or more datasets that you want to be used in experiments.

```
> projit add dataset train data/train.csv
```

This dataset can then be accessed inside any script by querying the project data as in this python example:

```
import projit as pit
project = pit.projit_load()
train_data_path = project.get_dataset("train")
```

DRAFT

Once your datasets are registered and available, you register experiments as follows.

```
> projit add experiment "Initial Exp" exp/exp_one.py
```

Alternatively, you can add the experiment within the script itself, as well as start and end its execution to track executions over time with this python example:

```
import projit as pit
project = pit.projit_load()
exec_id = project.start_experiment("Initial Exp", "exp/exp_one.py", params={})
# INSERT ALL EXPERIMENT CODE HERE
project.end_experiment("Initial Exp", exec_id, hyperparams={})
```

After experiments have been executed you want to register experimental results that are associated with the experiment. Follow this python example:

```
project.add_result("Initial Exp", <METRIC_NAME>, <NUMERIC_VALUE>, <DATASET_NAME>)
```

This function can be called multiple times for every metric you want to record. The dataset name is optional, and allows you to store metrics against different cuts of data for later analysis.

Finally you can use the command line tools to query and tabulate the results:

```
> projit list results <DATASET_NAME>
```


Acknowledgements

We acknowledge contributions from Jesse Wu and Priyabrata Karmakar in testing or reviewing the functionality and codebase of `projit`.

References

Alberti, M., Pondenkandath, V., Gygli, M., Ingold, R., & Liwicki, M. (2018). *DeepDIVA: A highly-functional python framework for reproducible experiments*.

Domenech, A. M., & Guillén, A. (2020). Ml-experiment: A python framework for reproducible data science. *Journal of Physics: Conference Series*, *1603*(1), 012025. https://doi.org/10.1088/1742-6596/1603/1/012025

Hawkins, J. (2023). What's in a domain? Anaylsis of URL features. *4th International Conference on Data Science and Cloud Computing (DSCC 2023)*, 85–93.

Hawkins, J., & Tivey, D. (2024, June). Literature filtering for systematic reviews with transformers. *2nd International Conference on Communications, Computing and Artificial Intelligence (CCCAI 2024)*. https://doi.org/https://doi.org/10.1145/3676581.3676582

Howe, B., Lawson, P., Bellinger, M. R., Anderson, E., Santos, E., Freire, J., Scheidegger, C., Baptista, A., & Silva, C. (2008). End-to-end eScience: Integrating workflow, query, visualization, and provenance at an ocean observatory. *Proceedings - 4th IEEE International Conference on eScience, eScience 2008*, 127–134. https://doi.org/10.1109/eScience.2008.67

Paolino, C. (2020). Cookiecutter modern data science. In *GitHub repository*. GitHub. https://github.com/crmne/cookiecutter-modern-datascience

Pettit, C., Russel, A. B. M., Michael, A., Aurambout, J.-P., Sharma, S., Williams, S., Hunter, D., Chan, P., Borda, A., Bishop, I., & Abramson, D. (2010). Realising an eScience platform to support climate change adaptation in victoria. *Proceedings - 2010 6th IEEE International Conference on e-Science, eScience 2010*, 73–80. https://doi.org/10.1109/eScience.2010.32

DRAFT

Subramanian, S., Sztromwasser, P., Puntervoll, P., & Petersen, K. (2013). Pipelined data-flow delegated orchestration for data-intensive eScience workflows. *International Journal of Web Information Systems*, *9*. https://doi.org/10.1108/IJWIS-05-2013-0012

Torvalds, L. (2005). Git. In *Git repository*. Software Freedom Conservancy. https://git.kernel.org/pub/scm/git/git.git/

Wirth, R., & Hipp, J. (2000). CRISP-DM: Towards a standard process model for data mining. *Proceedings of the 4th International Conference on the Practical Applications of Knowledge Discovery and Data Mining*, *1*, 29–39.

DRAFT